

A
Project report
On
“INSTITUTE MANAGEMENT SYSTEM”
Submitted in partial fulfillment of the requirements
for the award of the Degree of
Master of Science
In
Computer Science



Supervisor :
Gayatri Pundir
Assistant Professor

Submitted By :
Mohd Shuaib (M241560006)

Faculty of Computer Science & Information Technology
Motherhood University, Roorkee (UK)
Academic Session 2025 - 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Place: Motherhood University

Date:

Mohd Shuaib (M241560006)

CERTIFICATE

This is to certify that **Mohd Shuaib** student of the **Master of Science in Computer Science (4th Semester)**, have completed and submitted their project report entitled **“INSTITUTE MANAGEMENT SYSTEM”** under our guidance and supervision.

The project work carried out by them is a genuine and original contribution, completed during the academic session as a part of the curriculum requirements for the award of the degree. The students have demonstrated sincere effort, dedication, and a clear understanding of the subject matter throughout the development of this project.

To the best of our knowledge, this work has not been submitted to any other university or institution for the award of any degree or diploma. The project meets the required academic standards and is approved for submission.

Place : Motherhood University

Date :

Gayatri Pundir

Assistant Professor

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without complementing those who made it possible, whose guidance and encouragement made our efforts successful.

We express our sincere gratitude to **Dr. Ravindra Kumar** (Dean/HOD), **Motherhood University, Roorkee**, for providing the required facility.

I am extremely thankful to **Gayatri Pundir**, Assistant. Professor, Dept. of CS&IT, who helped us complete this project successfully by providing guidance, encouragement, and valuable suggestions throughout. I thank all our computer science staff and others who helped directly or indirectly to complete our project work with great success.

Finally, I am grateful to our parents and classmates for the invaluable support, guidance, and encouragement.

Mohd Shuaib (2416080002)
(M.Sc. (CS) IV Sem)

ABSTRACT

The **Institute Management System** is a software application designed to streamline and automate the day-to-day operations of an educational institution. It is beneficial for both students and administrative staff by reducing manual work and improving overall efficiency. In the traditional system, most activities such as maintaining student records, tracking attendance, and managing departmental data are performed manually. This approach is time-consuming, prone to human errors, and difficult to scale as the number of students increases. The proposed system addresses these challenges by providing an efficient, reliable, and scalable digital platform. The system allows users to register and access functionalities based on their roles. There are two main types of users in the system: Student and Administrator. The Administrator has full control over the system. The admin can add new users, manage student records, update student details, maintain attendance records, and organize data department-wise. This ensures proper data management and easy access to information. The Student can log in to the system to view their personal details and attendance status. By entering their respective roll numbers, students can quickly search and access their academic information. Overall, the Institute Management System simplifies institutional processes, reduces paperwork, ensures data accuracy, and enhances accessibility. It provides a centralized system for managing student-related information effectively, making it a valuable tool for modern educational institutions.

LIST OF FIGURE

- Figure 3.1 Entity Relationship Model (Page no. 6)
- Figure 3.2 Spiral Model (Page no. 6)
- Figure 5.1 Deployment architecture flow diagram (Page no. 43)
- Figure 6.1 Home Page (Page no. 49)
- Figure 6.2 Login Page (Page no. 49)
- Figure 6.3 Adding Student (Page no. 50)
- Figure 6.4 Adding Department (Page no. 50)
- Figure 6.5 Adding Attendance (Page no. 51)
- Figure 6.6 Trigger (Page no. 51)
- Figure 6.7 Student details (Page no. 52)
- Figure 6.8 Edit Student details (Page no. 52)
- Figure 6.9 Search Student details (Page no. 53)

LIST OF TABLE

Table 3.1 User table (Page no. 7)

Table 3.2 Student table (Page no. 7)

Table 3.3 Attendance table (Page no. 7)

Table 3.4 Department table (Page no. 7)

TABLE OF CONTENT

Declaration	I
Certificate	II
Acknowledgement	III
Abstract	IV
List of Figures	V
List of Tables	VI
Chapter 1: Introduction	1-2
1.1 Objectives	
1.2 Limitations	
Chapter 2: Study of Existing System	3-4
2.1 A Case Study	
2.2 Proposed System	
Chapter 3: Database Design	5-7
3.1 Software & Hardware Requirements	
3.2 Conceptual Design	
3.3 Tables created for database	
Chapter 4: Implementation	8-42
4.1 Backend	
4.2 Trigger	
4.3 Stored Procedure	
4.4 Frontend	
Chapter 5: Deployment	43-46
5.1 Deployment architecture	
5.2 Containerization	
5.3 Deployment on AWS EC2	
5.4 AI Involvement	
Chapter 6: Conclusion	47-53
6.1 Future Enhancements	
6.2 Screenshots	
References	54

CHAPTER-1

INTRODUCTION

In today's digital era, information technology plays a crucial role in transforming the way educational institutions function and manage their operations. With the continuous increase in the number of students, courses, and academic activities, managing institutional data through traditional manual methods has become increasingly challenging. Educational institutions generate large volumes of data related to student admissions, attendance, departmental activities, and academic records. Handling this information manually is not only time-consuming but also increases the chances of human errors, data redundancy, and mismanagement. Manual record-keeping systems that rely on paper registers, spreadsheets, and physical documentation make it difficult to maintain accuracy, ensure security, and retrieve information quickly when required. As institutions grow in size and complexity, an automated, centralized management system becomes essential.

To overcome these challenges, the Institute Management System has been developed as a comprehensive software solution designed to automate and streamline academic and administrative processes. The system aims to replace traditional manual methods with a modern digital platform that enhances efficiency, reliability, and data accessibility. The existing manual system used in many institutions suffers from several limitations, including difficulty maintaining records, slow information retrieval, a higher risk of data loss, and inefficient communication between departments. Managing attendance manually often results in incorrect entries and requires significant time and effort to maintain. Additionally, generating reports and summaries using manual processes is labor-intensive and prone to inaccuracies. The Institute Management System is a web-based software application designed to manage and organize institutional operations effectively through a centralized platform. It provides a structured interface for administrators and students to access and manage information based on their roles. The system automates core functionalities such as student information management, attendance tracking, departmental data handling, and secure data storage. By digitizing these processes, the system reduces paperwork, minimizes human errors, and improves overall operational efficiency.

1.1 OBJECTIVES:

The main objective of the project is to design and develop a user-friendly system.

Easy to use and an efficient computerized system.

- To develop an accurate and flexible system, it will eliminate data redundancy.
- To study the functioning of the Students' Management System.
- To make software fast in processing, with a good user interface.
- To provide a synchronized and centralized farmer and seller database.
- Computerization can be helpful as a means of saving time and money.
- To provide a better Graphical User Interface (GUI).
- Fewer chances of information leakage.
- Provides Security to the data by using login and password methods.
- To provide immediate storage and retrieval of data and information.
- Improving arrangements for student coordination.
- Reducing paperwork.
- To make software with a good user interface so that users can change it, and it should be used for a long time without errors and maintenance.

1.2 LIMITATIONS:

- Time consumption in data entry as the records are to be manually maintained by faculty members.
- A lot of paperwork is involved as the records are maintained in the files and registers.
- Storage requires as files and registers are used, the storage space requirement increases.
- Less Reliable use of papers for storing valuable data information is not at all reliable.
- Aadhar linkage with the official Aadhar database has not been done.

CHAPTER-2

STUDY OF EXISTING SYSTEM

2.1 CASE STUDY

The success of any organization, such as the School of Public Health, University of Delhi, depends on its ability to acquire accurate and timely data about its operations, manage this data effectively, and use it to analyze and guide its activities. Integrated student database systems offer users (Student, Registrar, HOD) a unified view of data from multiple sources. To provide a single consistent result for every object represented in these data sources, data fusion is concerned with resolving data inconsistency present in heterogeneous sources of data. The main objective of this project is to build a rigid and robust integrated student database system that will track and store records of students. This easy-to-use, integrated database application is geared towards reducing time spent on administrative tasks. The system is intended to accept processes and generate reports accurately, and any user can access the system at any point in time, provided the internet facility is available. The system is also intended to provide better services to users, provide meaningful, consistent, and timely data and information, and promote efficiency by converting paper-based processes into electronic form. The system was developed using technologies such as HTML, CSS, JavaScript, Python Flask, and MySQL. Python Flask is used for backend development, while HTML and CSS are used to build the user interface, and MySQL is used for database management. The system is designed to be efficient, reliable, and time-saving due to careful planning and implementation. All phases of the software development life cycle are employed, making the system robust and adaptable. In addition, the existing system highlights the importance of centralized data storage and accessibility. It ensures better coordination between departments and reduces duplication of data entries. The system also improves transparency and accountability in managing student information. However, despite these improvements, there are still challenges such as limited automation, dependency on manual inputs in some areas, and a lack of advanced analytical features. These limitations indicate the need for a more advanced and fully automated system.

2.2 PROPOSED SYSTEM

The proposed Institute Management System aims to overcome the limitations of the existing system by providing a fully automated, efficient, and user-friendly platform for managing institutional activities. It focuses on integrating all major functions such as student record management, attendance tracking, department management, and user authentication into a single system. The proposed system ensures that all data is stored in a structured and centralized database, which improves data consistency and reduces redundancy. It introduces role-based access control, allowing administrators and students to access only the relevant features as per their roles. The system is designed to handle large volumes of data efficiently, ensuring scalability and performance even as the number of users increases. Additionally, the system provides real-time access to information, enabling users to retrieve and update data instantly. Automated validation mechanisms are implemented to prevent errors such as duplicate entries and incorrect data inputs. The system also enhances security by implementing authentication and data protection measures. Furthermore, the proposed system is flexible and can be easily extended with additional modules such as fee management, examination handling, result processing, and report generation. It also supports better decision-making by providing accurate and up-to-date information to administrators. The use of modern web technologies ensures that the system is accessible, responsive, and easy to maintain. Overall, the proposed system aims to improve efficiency, reduce manual workload, enhance data accuracy, and provide a reliable solution for managing institutional operations. It lays the foundation for a smart and digital educational environment and can be further upgraded with advanced technologies such as cloud computing and artificial intelligence in the future.

CHAPTER 3

DATABASE DESIGN

3.1 Software Requirements & Hardware Requirements

Minimum System Configuration:

- Processor: 1.1 GHz or faster (Dual-core recommended)
- RAM: Minimum 2 GB (4 GB or more recommended for smooth performance)
- Storage:
 - At least 2.5 GB of free disk space
 - 5400 RPM HDD (SSD recommended for better performance)
- Display: 1366 × 768 resolution or higher
- Input Devices: Keyboard and Mouse

Web Browsers:

- Google Chrome (recommended)
- Mozilla Firefox
- Microsoft Edge / Internet Explorer

Operating System Compatibility

- Windows: Windows 7, 8.1, 10, or later
- Linux: Ubuntu, Fedora, or other distributions
- macOS: macOS 10.12 or later

3.2 Conceptual Design

ER Diagram

The Entity-Relationship (ER) diagram represents the structure and relationships of the student management system. It consists of four main entities: **User**, **Student**, **Department**, and **Attendance**.

- The **User** entity stores login-related details such as username, email, and password. It has relationships that allow adding departments and managing student data.
- The **Student** entity contains detailed information about students, including student ID, name, roll number, semester, branch, contact details, and gender.
- The **Department** entity includes attributes like department ID and branch, representing different academic departments.
- The **Attendance** entity records attendance-related information such as attendance ID and roll number.

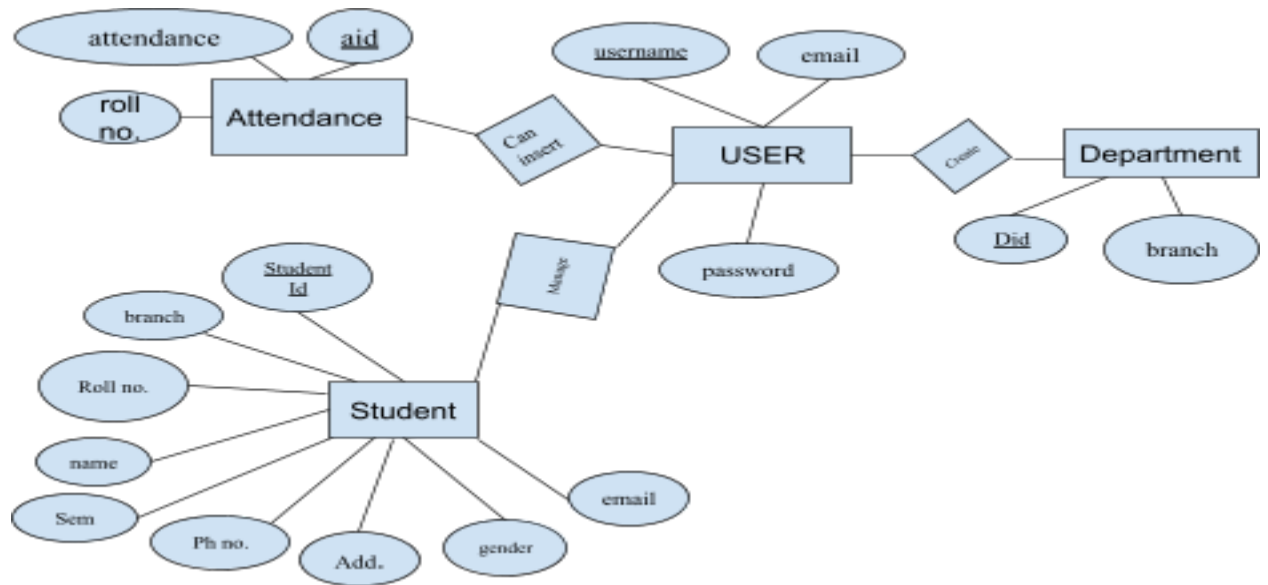


Figure 3.1 Entity Relationship Model

3.2.2 Software Development Model

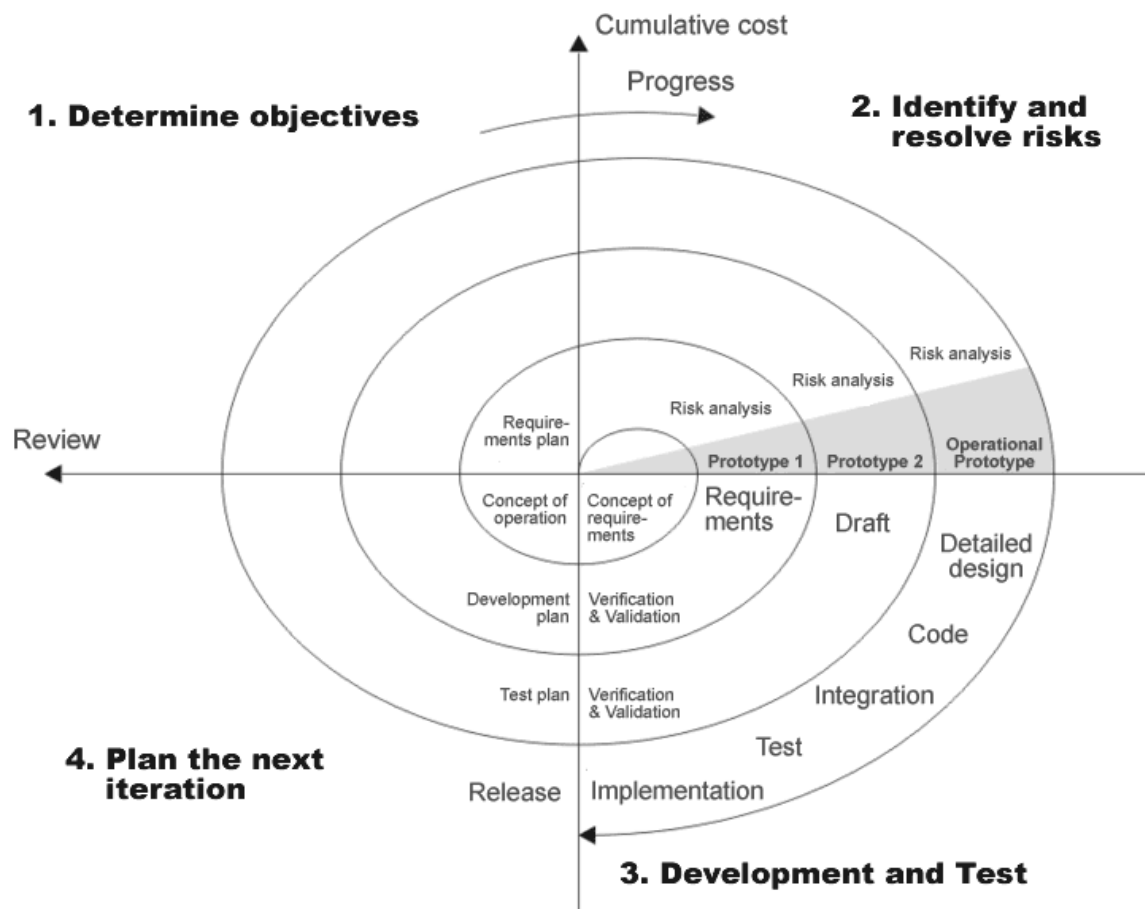


Figure 3.2 Spiral Model

Use of the Spiral Model in Development

The project followed the **Spiral Model**, which is a risk-driven process model combining elements of both iterative development and systematic aspects of the waterfall model. The Spiral Model was particularly beneficial for this project as it allowed continuous refinement through repeated cycles (or spirals), each consisting of planning, risk analysis, development, and evaluation phases.

3.3 Tables Created for Database

User

Fields	Data type	Description
username (Primary)	varchar(30)	Unique username
email	varchar(30)	User email
password	varchar(30)	User password

Table 3.1 User table

Student

Fields	Data type	Description
SID (primary)	int(30)	Unique student ID
Roll no.	int(30)	Student roll number
name	varchar(30)	Student name
Phone no.	int(30)	Student phone no.
Address	varchar(30)	Student's address
Branch	varchar(30)	Student's branch
Gender	varchar(30)	Student's gender
Sem	int(30)	semester

Table 3.2 Student table

Attendance

Fields	Data type	Description
Aid (Primary)	int(30)	Unique id
Attendance	int(30)	Attendance in percentage
Roll no.	int(30)	Student's roll no.

Table 3.3 Attendance table

Department

Fields	Data type	Description
Did	Int (30)	Unique id
Branch	varchar(30)	branch name

Table 3.4 Department table

CHAPTER 4

IMPLEMENTATION

An "implementation" of Python should be taken to mean a program or environment that provides support for the execution of programs written in the Python language, as represented by the C Python reference implementation.

There have been and are several distinct software packages providing what we all recognize as Python, although some of those are more like distributions or variants of some existing implementation than a completely new implementation of the language.

4.1 Back End

Database (MySQL) :

A Database Management System (DBMS) is computer software designed for the purpose of managing databases, a large set of structured data, and running operations on the data requested by numerous users. Typical examples of DBMSs include Oracle, DB2, Microsoft Access, Microsoft SQL Server, Firebird, PostgreSQL, MySQL, SQLite, FileMaker, and Sybase Adaptive Server Enterprise. DBMSs are typically used by Database administrators in the creation of Database systems. Typical examples of DBMS use include accounting, human resources, and customer support systems. Originally found only in large companies with the computer hardware needed to support large data sets, DBMSs have more recently emerged as a fairly standard part of any company's back office.

A DBMS is a complex set of software programs that controls the organization, storage, management, and retrieval of data in a database. A DBMS includes:

- A modeling language to define the schema of each database hosted in the DBMS, according to the DBMS data model.
- The dominant model in use today is the ad hoc one embedded in SQL, despite the objections of purists who believe this model is a corruption of the relational model, since it violates several of its fundamental principles for the sake of practicality and performance. Many DBMSs also support the Open Database Connectivity API, which supports a standard way for programmers to access the DBMS.
- Data structures (fields, records, files, and objects) optimized to deal with very large amounts of data stored on a permanent data storage device (which implies relatively slow access compared to volatile main memory). A database query

language and report Students Management System 7 writer to allow users to interactively interrogate the database, analyze its data, and update it according to the users' privileges on data.

- Data security prevents unauthorized users from viewing or updating the database. Using passwords, users are allowed access to the entire database or subsets of it called sub-schemas. For example, an employee database can contain all the data about an individual employee, but one group of users may be authorized to view only payroll data, while others are allowed access to only work history and student data.
- If the DBMS provides a way to interactively enter and update the database, as well as interrogate it, this capability allows for managing personal databases. However, it may not leave an audit trail of actions or provide the kinds of controls necessary in a multi-user organization. These controls are only available when a set of application programs is customized for each data entry and updating function.

A transaction mechanism that ideally would guarantee the ACID properties, in order to ensure data integrity, despite concurrent user accesses (concurrency control), and faults (fault tolerance).

- It also maintains the integrity of the data in the database. The DBMS can maintain the integrity of the database by not allowing more than one user to update the same record at the same time.
- The DBMS can help prevent duplicate records via unique index constraints; for example, no two customers with the same customer numbers (key fields) can be entered into the database. See ACID properties for more information (Redundancy avoidance).

When a DBMS is used, information systems can be changed much more easily as the organization's information requirements change. Organizations may use one kind of DBMS for daily transaction processing and then move the details onto another computer that uses another DBMS better suited for random inquiries and analysis. Overall system design decisions are performed by data administrators and systems analysts. Detailed database design is performed by database administrators.

SQL:

Structured Query Language (SQL) is the language used to manipulate relational databases. SQL is tied very closely to the relational model. In the relational model, data is stored in structures called relations or tables. SQL statements are issued for:

Data Definition: Defining tables and structures in the database (DDL is used to create, alter, and drop schema objects such as tables and indexes).

Data Manipulation: Used to retrieve and modify data stored in the database (DML includes commands like SELECT, INSERT, UPDATE, and DELETE).

Data Control: Used to control access to data in the database (DCL includes commands like GRANT and REVOKE).

Transaction Control: Used to manage transactions in the database (TCL includes commands like COMMIT, ROLLBACK, and SAVEPOINT).

BACKEND PYTHON WITH MYSQL CODE

```
from flask import Flask,render_template,request,session,redirect,url_for,flash
from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin
from Werkzeug.security import generate_password_hash,check_password_hash
from flask_login import login_user,logout_user,login_manager,LoginManager
from flask_login import login_required,current_user
import json
# MY db connection
local_server= True
app = Flask(__name__)
app.secret_key='kusumachandashwini'
# this is for getting unique user access
login_manager=LoginManager(app)
login_manager.login_view='login'
@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))
#
app.config['SQLALCHEMY_DATABASE_URL']='mysql://username:password@localho
```

```

st/databas_table_                                     name'
app.config['SQLALCHEMY_DATABASE_URI']='mysql://root:@localhost/students'
db=SQLAlchemy(app)
# Here we will create DB models that are tables
class Test(db.Model):
    id=db.Column(db.Integer,primary_key=True)
    name=db.Column(db.String(100))
    email=db.Column(db.String(100))
class Department(db.Model):
    cid=db.Column(db.Integer,primary_key=True)
    branch=db.Column(db.String(100))
class Attendance(db.Model):
    aid=db.Column(db.Integer,primary_key=True)
    rollno=db.Column(db.String(100))
    attendance=db.Column(db.Integer())
class Trig(db.Model):
    tid=db.Column(db.Integer,primary_key=True)
    rollno=db.Column(db.String(100))
    action=db.Column(db.String(100))
    timestamp=db.Column(db.String(100))
class User(UserMixin,db.Model):
    id=db.Column(db.Integer,primary_key=True)
    username=db.Column(db.String(50))
    email=db.Column(db.String(50),unique=True)
    password=db.Column(db.String(1000))
class Student(db.Model):
    id=db.Column(db.Integer,primary_key=True)
    rollno=db.Column(db.String(50))
    sname=db.Column(db.String(50))
    sem=db.Column(db.Integer)
    gender=db.Column(db.String(50))
    branch=db.Column(db.String(50))
    email=db.Column(db.String(50))
    number=db.Column(db.String(12))

```

```

        address=db.Column(db.String(100))

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/studentdetails')
def studentdetails():
    query=db.engine.execute(f"SELECT * FROM `student`")
    return render_template('studentdetails.html',query=query)

@app.route('/triggers')
def triggers():
    query=db.engine.execute(f"SELECT * FROM `trig`")
    return render_template('triggers.html',query=query)

@app.route('/department',methods=['POST','GET'])
def department():
    if request.method=="POST":
        dept=request.form.get('dept')
        query=Department.query.filter_by(branch=dept).first()
        if query:
            flash("Department Already Exist", "warning")
            return redirect('/department')
        dep=Department(branch=dept)
        db.session.add(dep)
        db.session.commit()
        flash("Department Added", "success")
    return render_template('department.html')

@app.route('/addattendance',methods=['POST','GET'])
def addattendance():
    query=db.engine.execute(f"SELECT * FROM `student`")
    if request.method=="POST":
        rollno=request.form.get('rollno')
        attend=request.form.get('attend')
        print(attend,rollno)
        atte=Attendance(rollno=rollno,attendance=attend)
        db.session.add(atte)

```

```

        db.session.commit()

        flash("Attendance added", "warning")
    return render_template('attendance.html', query=query)

@app.route('/search', methods=['POST', 'GET'])
def search():
    if request.method == "POST":
        rollno = request.form.get('roll')
        bio = Student.query.filter_by(rollno=rollno).first()
        attend = Attendance.query.filter_by(rollno=rollno).first()
        return render_template('search.html', bio=bio, attend=attend)
    return render_template('search.html')

@app.route("/delete/", methods=['POST', 'GET'])
@login_required
def delete(id):
    db.engine.execute(f"DELETE FROM `student` WHERE `student`.`id`={id}")
    flash("Slot Deleted Successful", "danger")
    return redirect('/studentdetails')

@app.route("/edit/", methods=['POST', 'GET'])
@login_required
def edit(id):
    dept = db.engine.execute("SELECT * FROM `department`")
    posts = Student.query.filter_by(id=id).first()
    if request.method == "POST":
        rollno = request.form.get('rollno')
        sname = request.form.get('sname')
        sem = request.form.get('sem')
        gender = request.form.get('gender')
        branch = request.form.get('branch')
        email = request.form.get('email')
        num = request.form.get('num')
        address = request.form.get('address')
        query = db.engine.execute(f"UPDATE `student` SET
`rollno`='{rollno}', `sname`='{sname}', `sem`='{sem}', `gender`='{gender}', `branch`
`='{branch}', `email`='{email}', `number`='{num}', `address`='{address}'")

```

```

        flash("Slot is updated", "success")
        return redirect('/studentdetails')
return render_template('edit.html',posts=posts,dept=dept)
@app.route('/signup',methods=['POST','GET'])
def signup():
    if request.method == "POST":
        username=request.form.get('username')
        email=request.form.get('email')
        password=request.form.get('password')
        user=User.query.filter_by(email=email).first()
        if user:
            flash("Email Already Exist", "warning")
            returnrender_template('/signup.html')
        encpassword=generate_password_hash(password)
        new_user=db.engine.execute(f'INSERT INTO `user` (`username`,
`email`,` password`) VALUES ('{username}','{email}','{encpassword}')')
        # this is method 2 to save data in db
        #
        newuser=User(username=username,email=email,password=encpassword)
        # db. session.add(newuser)
        # db. session.commit()
        flash("Signup Success Please Login","success")
        return render_template('login.html')
        return render_template('signup.html')
@app.route('/login',methods=['POST','GET'])
def login():
    if request.method == "POST":
        email=request.form.get('email')
        password=request.form.get('password')
        user=User.query.filter_by(email=email).first()
        if user and check_password_hash(user.password,password):
            login_user(user)
            flash("Login Success", "primary")
            return redirect(url_for('index'))

```

```

        else:
            flash("invalid credentials", "danger")
            return render_template('login.html')
    return render_template('login.html')

@app.route('/logout')
@login_required
def logout():
    logout_user()
    flash("Logout Successful", "warning")
    return redirect(url_for('login'))

@app.route('/addstudent',methods=['POST','GET'])
@login_required
def addstudent():
    dept=db.engine.execute("SELECT * FROM `department`")
    if request.method=="POST":
        rollno=request.form.get('rollno')
        sname=request.form.get('sname')
        sem=request.form.get('sem')
        gender=request.form.get('gender')
        branch=request.form.get('branch')
        email=request.form.get('email')
        num=request.form.get('num')
        address=request.form.get('address')
        query=db.engine.execute(f'INSERT INTO `student` (`rollno`, `sname`,
        `sem`, `gender`, `branch`, `email`, `number`, `address`) VALUES
        ({rollno},{sname},{sem},{gender},{branch},{email},{num},{address})')
        flash("Booking Confirmed", "info")
    return render_template('student.html',dept=dept)

@app.route('/test')
def test():
    try:
        Test.query.all()
        return 'My database is connected.'
    except:

```

```
        return 'My db is not connected.'  
app.run(debug=True)
```

4.2: Triggers

It is a special kind of stored procedure that automatically executes when an event occurs in the database.

Triggers used :

1: Trigger name: on insert

Table: register

Time: after

Event: insert

```
INSERT INTO trig VALUES(null,NEW.rid,'Farmer Inserted',NOW())
```

2: Trigger name: on delete

Table: register

Time: after

Event: delete Definition:

```
INSERT INTO trig VALUES(null,OLD.rid,'FARMER DELETED',NOW())
```

3: Trigger name: on update

Table: register

Time: after

Event: update Definition:

```
INSERT INTO trig VALUES(null,NEW.rid,'FARMER UPDATED',NOW())
```

4.3: Stored Procedure

Routine name: proc

Type: procedure

Definition: Select * from register;

4.4 FRONTEND

LOGIN

```
{% extends 'auth.html' %}
{% block title %}
login
{% endblock title %}
{% block content %}
<h1 class="mb-4 pb-0"><span>Login</span> </h1>
{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
<form class="text-white py-3 px-3" action="/login" method="post">
    <div class="form-group">
        <label for="exampleInputEmail">Email address</label>
        <input type="email" class="form-control mb-3" id="exampleInputEmail"
name="email" aria-describedby="emailHelp" required>
    </div>
    <div class="form-group">
        <label for="exampleInputPassword1">Password</label>
        <input type="password" class="form-control" id="exampleInputPassword1"
name="password" required>
    </div><br>
    <button type="submit" class="form-control bg-danger text-white">Login</button>
    <p class="mb-2 pb-0">New User?</p>
    <a href="/signup" class="about-btn scrollTo">Signup</a>
</form>
{% endblock content %}
```

SIGNUP

```
{% extends 'auth.html' %}
{% block title %}
Signup
{% endblock title %}
{% block content %}
<h1 class="mb-1"><span>Sign Up</span> </h1>
{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
<form class="text-white py-3 px-3" action="/signup" method="post">
<div class="form-group">
    <label for="exampleInputPassword1">UserName</label>
    <input type="text" class="form-control mb-2" name="username" id="username"
        required>
</div>
<div class="form-group">
    <label for="exampleInputEmail1">Email address</label>
    <input type="email" class="form-control mb-2" id="exampleInputEmail1"
        name="email" aria-describedby="emailHelp" required>
    <small id="emailHelp" class="form-text text-muted">We'll never share your email with
Anyone else.</small>
</div>
<div class="form-group">
    <label for="exampleInputPassword1">Password</label>
    <input type="password" class="form-control mb-2" name="password"
        id="exampleInputPassword1" required>
</div>
```

```

<br>
<button type="submit" class="form-control bg-danger text-white">Sign In</button>
<p class="mb-2 pb-0">Already a User?</p>
<a href="/login" class="about-btn scrollto">Login</a>
</form>
{% endblock content %}

```

INDEX

```

{% extends 'base.html' %}
{% block title %}
Student
{% endblock title %}
{% block home %}
menu-active
{% endblock home %}
{% block body %}
<!-- ===== Footer ===== -->
<footer id="footer">
<div class="footer-top">
</div>
<div class="container">
<div class="credits">
<br>
Developed by <a href="/">M.Sc. Computer Science Motherhood University
Roorkee</a>
</div>
</div>
</footer><!-- End Footer -->
{% endblock body %}

```

BASIC

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="utf-8">
<meta content="width=device-width, initial-scale=1.0" name="viewport">
<title>{% block title %}
{% endblock title %}</title>
<meta content="" name="description">
<meta content="" name="keywords">
<linkhref="https://fonts.googleapis.com/css?family=Open+Sans:300,300i,400,400i,700,700i|Raleway:300,400,500,700,800" rel="stylesheet">
<!-- Vendor CSS Files -->
<link href="static/assets/vendor/bootstrap/css/bootstrap.min.css" rel="stylesheet">
<link href="static/assets/vendor/venobox/venobox.css" rel="stylesheet">
<linkhref="static/assets/vendor/font-awesome/css/font-awesome.min.css"
rel="stylesheet">
<linkhref="static/assets/vendor/owl.carousel/assets/owl.carousel.min.css"rel="stylesheet">
<link href="static/assets/vendor/aos/aos.css" rel="stylesheet">
<!-- Template Main CSS File -->
<link href="static/assets/css/style2.css" rel="stylesheet">
{% block style %}
{% endblock style %}
</head>
<body>
<!-- ===== Header ===== -->
<header id="header">
<div class="container">
<div id="logo" class="pull-left">
<a href="/" class="scrollto">Institute Management System</a>
</div>
<nav id="nav-menu-container">
<ul class="nav-menu">
<li class="{% block home %} {% endblock home %}">
<a href="/">Home</a></li>
<li><a href="/about">About</a></li>
<li><a href="/contact">Contact</a></li>
{% if user.is_authenticated %}

```

```

<li class="buy-tickets"><a href="">Welcome</a></li>
<li class="buy-tickets"><a href="/logout">Logout</a></li>
{% else %}
<li class="buy-tickets"><a href="/signup">Sign Up</a></li>
{% endif %}
</ul>
</nav><!-- #nav-menu-container -->
</div>
</header><!-- End Header -->
<!-- ===== Intro Section ===== -->
{% block intro %}
{% endblock intro %}
<main id="main">
{% block body %}
{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
{{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
{% endblock body %}
<!-- Vendor JS Files -->
<script src="static/assets/vendor/jquery/jquery.min.js"></script>
<script src="static/assets/vendor/bootstrap/js/bootstrap.bundle.min.js"></script>
<script src="static/assets/vendor/jquery.easing/jquery.easing.min.js"></script>
<script src="static/assets/vendor/php-email-form/validate.js"></script>
<script src="static/assets/vendor/venobox/venobox.min.js"></script>
<script src="static/assets/vendor/owl.carousel/owl.carousel.min.js"></script>
<script src="static/assets/vendor/superfish/superfish.min.js"></script>
<script src="static/assets/vendor/hoverIntent/hoverIntent.js"></script>
<script src="static/assets/vendor/aos/aos.js"></script>

```

```

<!-- Template Main JS File -->
<script src="static/assets/js/main.js"></script>
</body>
</html>

```

STUDENT

```

{% extends 'base.html' %}
{% block title %}
Add Students
{% endblock title %}
{% block body %}
<h3 class="text-center"><span>Add Student Details</span> </h3>
{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
<br>
<div class="container">
<div class="row">
<div class="col-md-4"></div>
<div class="col-md-4">
<form action="/addstudent" method="post">
<div class="form-group">
<label for="rollno">Roll Number</label>
<input type="text" class="form-control" name="rollno" id="rollno">
</div>
<br>
<div class="form-group">
<label for="sname">Student Name</label>

```

```

<input type="text" class="form-control" name="sname" id="sname">
</div>
<br>
<div class="form-group">
<label for="sem">Sem</label>
<input type="number" class="form-control" name="sem" id="sem">
</div>
<br>
<div class="form-group">
<select class="form-control" id="gender" name="gender" required>
<option selected>Select Gender</option>
<option value="male">Male</option>
<option value="female">Female</option>
</select>
</div>
<br>
<div class="form-group">
<select class="form-control" id="branch" name="branch" required>
<option selected>Select Branch</option>
    {% for d in dept %}
    <option value="{{d.branch}}">{{d.branch}}</option>
    {% endfor %}
</select>
</div>
<br>
<div class="form-group">
<label for="email">Email</label>
<input type="email" class="form-control" name="email" id="email">
</div>
<br>
<div class="form-group">
<label for="num">Phone Number</label>
<input type="number" class="form-control" name="num" id="num">
</div>

```

```

<br>
<div class="form-group">
<label for="address">Address</label>
<textarea class="form-control" name="address" id="address"></textarea>
</div>
<br>
    <button type="submit" class="btn btn-danger btn-sm btn-block">Add Record</button>
</form>
<br>
<br>
</div>
<div class="col-md-4"></div>
</div></div>
{% endblock body %}

```

DEPARTMENT

```

{% extends 'base.html' %}
{% block title %}
Add Department
{% endblock title %}
{% block body %}
<h3 class="text-center bg-danger text-white"><span>Add Department</span> </h3>
{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
<br>
<div class="container">
<div class="row">

```



```

<div class="col-md-4"></div>
<div class="col-md-4">
<form action="/department" method="post">
<div class="form-group">
<label for="dept">Enter Department</label>
<input type="text" class="form-control" name="dept" id="dept">
</div>
<br>
<button type="submit" class="btn btn-danger btn-sm btn-block">Add
Department</button>
</form>
<br>
<br>
</div>
<div class="col-md-4"></div>
</div></div>
{% endblock body %}

```

STUDENT DETAILS

```

{% extends 'base.html' %}
{% block title %}
Student Details
{% endblock title %}
{% block body %}
<h3 class="text-center"><span>Student Details</span> </h3>
{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}

```

```

<br>
<table class="table">
  <thead class="thead-light">
    <tr>
      <th scope="col">SID</th>
      <th scope="col">ROLL NUMBER</th>
      <th scope="col">STUDENT NAME</th>
      <th scope="col">SEM</th>
      <th scope="col">GENDER</th>
      <th scope="col">BRANCH</th>
      <th scope="col">EMAIL</th>
      <th scope="col">NUMBER</th>
      <th scope="col">ADDRESS</th>
      <th scope="col">EDIT</th>
      <th scope="col">DELETE</th>
    </tr>
  </thead>
  <tbody>
    {% for post in query %}
      <tr>
        <th scope="row">{{post.id}}</th>
        <td>{{post.rollno}}</td>
        <td>{{post.sname}}</td>
        <td>{{post.sem}}</td>
        <td>{{post.gender}}</td>
        <td>{{post.branch}}</td>
        <td>{{post.email}}</td>
        <td>{{post.number}}</td>
        <td>{{post.address}}</td>
        <td><a href="/edit/{{post.id}}"><button class="btn btn-primary">Edit </button> </a>
      </td>
        <td><a href="/delete/{{post.id}}"><button onclick="return confirm('Are you sure to
          Delete data');" class="btn btn-danger">Delete </button> </a> </td>
      </tr>
    {% endfor %}
  </tbody>
</table>

```

```
{% endfor %}
    </tbody>
</table>
{% endblock body %}
```

BASE

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta content="width=device-width, initial-scale=1.0" name="viewport">
    <title>{% block title %}
    {% endblock title %}</title>
    <meta content="" name="description">
    <meta content="" name="keywords">
    {% block style %}
    {% endblock style %}
    <link href="https://fonts.googleapis.com/css?family=Open+Sans:300,300i,400,400i,700,700i|Raleway:300,400,500,700,800" rel="stylesheet">
    <!-- Vendor CSS Files -->
    <link href="static/assets/vendor/bootstrap/css/bootstrap.min.css" rel="stylesheet">
    <link href="static/assets/vendor/venobox/venobox.css" rel="stylesheet">
        <link href="static/assets/vendor/font-awesome/css/font-awesome.min.css"
        rel="stylesheet">
        <link href="static/assets/vendor/owl.carousel/assets/owl.carousel.min.css"
        rel="stylesheet">
    <link href="static/assets/vendor/aos/aos.css" rel="stylesheet">
    <!-- Template Main CSS File -->
    <link href="static/assets/css/style.css" rel="stylesheet">
</head>
<body>
    <!-- ===== Header ===== -->
    <header id="header">
        <div class="container">
```

```

<div id="logo" class="pull-left">
  <a href="/" class="scrollto">Institute Management System</a>
</div>

<nav id="nav-menu-container">
  <ul class="nav-menu">
    <li class="{% block home %}
      {% endblock home %}"><a href="/">Home</a></li>
    <li><a href="/addstudent">Students</a></li>
    <li><a href="/addattendance">Attendance</a></li>
    <li><a href="/department">Department</a></li>
    <li><a href="/triggers">Records</a></li>
    <li><a href="/studentdetails">Student Details</a></li>
    <li><a href="/search">Search</a></li>
    <!-- <li><a href="/about">About</a></li> -->
    {% if current_user.is_authenticated %}
      <li class="buy-tickets"><a href="">Welcome {{current_user.username}}</a></li>
      <li class="buy-tickets"><a href="/logout">Logout</a></li>
    {% else %}
      <li class="buy-tickets"><a href="/signup">Sign Up</a></li>
    {% endif %}
  </ul>
</nav><!-- #nav-menu-container -->
</div>

</header><!-- End Header -->

<!-- ===== Intro Section ===== -->

<section id="intro">
  <div class="intro-container" data-aos="zoom-in" data-aos-delay="100">
    <h1 class="mb-4 pb-0">INSTITUTE MANAGEMENT SYSTEM </span> </h1>
    <p class="mb-4 pb-0">Manage your student's information here.</p>
    <a href="" class="about-btn scrollto">View More</a>
  </div>
</section><!-- End Intro Section -->

<main id="main">
  {% block body %}

```

```

{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
{% endblock body %}
<a href="#" class="back-to-top"><i class="fa fa-angle-up"></i></a>
<!-- Vendor JS Files -->
<script src="static/assets/vendor/jquery/jquery.min.js"></script>
<script src="static/assets/vendor/bootstrap/js/bootstrap.bundle.min.js"></script>
<script src="static/assets/vendor/jquery.easing/jquery.easing.min.js"></script>
<script src="static/assets/vendor/php-email-form/validate.js"></script>
<script src="static/assets/vendor/venobox/venobox.min.js"></script>
<script src="static/assets/vendor/owl.carousel/owl.carousel.min.js"></script>
<script src="static/assets/vendor/superfish/superfish.min.js"></script>
<script src="static/assets/vendor/hoverIntent/hoverIntent.js"></script>
<script src="static/assets/vendor/aos/aos.js"></script>
<!-- Template Main JS File -->
<script src="static/assets/js/main.js"></script>
</body>
</html>

```

SEARCH

```

{% extends 'base.html' %}
{% block title %}
Search
{% endblock title %}
{% block body %}
<h3 class="text-center"><span>Search Your Details</span> </h3>
{% with messages=get_flashed_messages(with_categories=true) %}

```

```

{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
<br>
<div class="container">
<div class="row">
<div class="col-md-4">
<form action="/search" method="post">
<div class="form-group">
<label for="attendance">Enter Your Roll Number</label>
<input type="text" class="form-control" name="roll" id="roll" placeholder="Use Small
    Letter">
</div>
<br>
    <button type="submit" class="btn btn-danger btn-sm btn-block">Search</button>
</form>
<br>
<br>
</div>
<div class="col-md-8">
{% if bio %}
<br>
<h3>Your Details</h3>
<hr>
<li>Roll No : {{bio.rollno}}</li>
<li>Name : {{bio.sname}}</li>
<li>Sem : {{bio.sem}}</li>
<li>Gender : {{bio.gender}}</li>
<li>Branch : {{bio.branch}}</li>

```

```

<li>Email : {{bio.email}}</li>
<li>Number : {{bio.number}}</li>
<li>Address : {{bio.address}}</li>
<hr>
<h3>Attendance Status</h3>
<hr>
<li>Attendance : {{attend.attendance}}</li>
<br><br>
{% else %}
No such results
{% endif %}
</div>
</div>
{% endblock body %}

```

AUTH

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta content="width=device-width, initial-scale=1.0" name="viewport">
  <title>{% block title %}
  {% endblock title %}</title>
  <meta content="" name="description">
  <meta content="" name="keywords">
  {% block style %}
  {% endblock style %}
  <link href="https://fonts.googleapis.com/css?family=Open+Sans:300,300i,400,400i,700,700i|Raleway:300,400,500,700,800" rel="stylesheet">
  <!-- Vendor CSS Files -->
  <link href="static/assets/vendor/bootstrap/css/bootstrap.min.css" rel="stylesheet">
  <link href="static/assets/vendor/venobox/venobox.css" rel="stylesheet">
  <link href="static/assets/vendor/font-awesome/css/font-awesome.min.css"
  rel="stylesheet">

```

```

        <link href="static/assets/vendor/owl.carousel/assets/owl.carousel.min.css"
rel="stylesheet">
<link href="static/assets/vendor/aos/aos.css" rel="stylesheet">
<!-- Template Main CSS File -->
<link href="static/assets/css/style.css" rel="stylesheet">
</head>
<body>
<!-- ===== Header ===== -->
<header id="header">
<div class="container">
<div id="logo" class="pull-left">
<a href="/" class="scrollto">Institute Management System</a>
</div>
</div>
</header><!-- End Header -->
<!-- ===== Intro Section ===== -->
<section id="intro">
<div class="intro-container" data-aos="zoom-in" data-aos-delay="100">
{% block content %}
{% endblock content %}
</div>
</section><!-- End Intro Section -->
<div class="main">
<!-- ===== Footer ===== -->
<footer id="footer">
<div class="footer-top">
<div class="container">
</div>
<div class="container">
<div class="credits">
<br>
Developed by <a>M.Sc. Computer Science Motherhood University Roorkee</a>
</div>
</div>

```



```

</footer><!-- End Footer -->
<a href="#" class="back-to-top"><i class="fa fa-angle-up"></i></a>
<!-- Vendor JS Files -->
<script src="static/assets/vendor/jquery/jquery.min.js"></script>
<script src="static/assets/vendor/bootstrap/js/bootstrap.bundle.min.js"></script>
<script src="static/assets/vendor/jquery.easing/jquery.easing.min.js"></script>
<script src="static/assets/vendor/php-email-form/validate.js"></script>
<script src="static/assets/vendor/venobox/venobox.min.js"></script>
<script src="static/assets/vendor/owl.carousel/owl.carousel.min.js"></script>
<script src="static/assets/vendor/superfish/superfish.min.js"></script>
<script src="static/assets/vendor/hoverIntent/hoverIntent.js"></script>
<script src="static/assets/vendor/aos/aos.js"></script>
<!-- Template Main JS File -->
<script src="static/assets/js/main.js"></script>
</body>
</html>

```

EDITS

```

<!doctype html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
      <meta name="viewport" content="width=device-width, initial-scale=1,
      shrink-to-fit=no">
  <!-- Bootstrap CSS -->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@4.5.3/dist/css/bootstrap.min.css" integrity="sha384-TX8t27EcRE3e/ihU7zmQxVncDAy5uIKz4rEkgIXeMed4M0jlfIDPvg6uqKI2xXr2" crossorigin="anonymous">
  <title>Edit</title>
</head>
<body class="bg-danger">
  <h3 class="text-center"><span>Edit Student Details</span> </h3>
  {% with messages=get_flashed_messages(with_categories=true) %}

```

```

{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
<br>
<div class="container">
<div class="row">
<div class="col-md-4"></div>
<div class="col-md-4">
<form action="/edit/{{posts.id}}" method="post">
<div class="form-group">
<label for="rollno">Roll Number</label>
<input type="text" class="form-control" name="rollno" id="rollno"
    value="{{posts.rollno}}" >
</div>
<br>
<div class="form-group">
<label for="sname">Student Name</label>
<input type="text" class="form-control" name="sname" id="sname"
    value="{{posts.sname}}" >
</div>
<br>
<div class="form-group">
<label for="sem">Sem</label>
<input type="number" class="form-control" name="sem" id="sem"
    value="{{posts.sem}}" >
</div>
<br>
<div class="form-group">
<select class="form-control" id="gender" name="gender" required>

```

```

    <option selected>{{posts.gender}}</option>
    <option value="male">Male</option>
    <option value="female">Female</option>
  </select>
</div>
<br>
<div class="form-group">
  <select class="form-control" id="branch" name="branch" required>
    <option selected>{{posts.branch}}</option>
    {% for d in dept %}
    <option value="{{d.branch}}">{{d.branch}}</option>
    {% endfor %}
  </select>
</div>
<br>
<div class="form-group">
  <label for="email">Email</label>
  <input type="email" class="form-control" name="email" id="email"
    value="{{posts.email}} ">
</div>
<br>
<div class="form-group">
  <label for="num">Phone Number</label>
  <input type="number" class="form-control" name="num" id="num"
    value="{{posts.number}} ">
</div>
<br>
<div class="form-group">
  <label for="address">Address</label>
  <input type="text" class="form-control" name="address" id="address"
    value="{{posts.address}} ">
</div>
<br>
  <button type="submit" class="btn btn-light btn-sm btn-block">Submit Record</button>

```

```

</form>
<br>
<br>
</div>
<div class="col-md-4"></div>
</div>
</div>
</div>

<!-- Optional JavaScript; choose one of the two! -->
<!-- Option 1: jQuery and Bootstrap Bundle (includes Popper) -->
    <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"
    integrity="sha384-DfXdz2htPH0lsSSs5nCTpuj/zy4C+OGpamoFVy38MVBnE+Ib
    bVYUew+OrCXaRkfj" crossorigin="anonymous"></script>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@4.5.3/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-ho+j7jyWK8fNQe+A12Hb8AhRq26LrZ/JpcUGGOn+Y7Rswe
    NrtN/tE3MoK7ZeZDyx" crossorigin="anonymous"></script>
<!-- Option 2: jQuery, Popper.js, and Bootstrap JS
    <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"
    integrity="sha384-DfXdz2htPH0lsSSs5nCTpuj/zy4C+OGpamoFVy38MVBnE+Ib
    bVYUew+OrCXaRkfj" crossorigin="anonymous"></script>
    <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js"
    integrity="sha384-9/reFTGAW83EW2RDu2S0VKA1Zap3H66lZ81PoYlFhbGU
    +6BZp6G7niu735Sk7lN" crossorigin="anonymous"></script>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@4.5.3/dist/js/bootstrap.min.js"
    integrity="sha384-w1Q4orYjBQndcko6MimVbzY0tgp4pWB4lZ7lr30WKz0vr/a
    WKhXDBNmNb5D92v7s" crossorigin="anonymous"></script>
-->
</body>
</html>

```

ATTENDANCE

{% extends 'base.html' %}

{% block title %}

Add Attendance

```

{% endblock title %}
{% block body %}
<h3 class="text-center"><span>Add Attendance Details</span> </h3>
{% with messages=get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for category, message in messages %}
<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">
    {{message}}
</div>
{% endfor %}
{% endif %}
{% endwith %}
<br>
<div class="container">
<div class="row">
<div class="col-md-4"></div>
<div class="col-md-4">
<form action="/addattendance" method="post">
<div class="form-group">
<select class="form-control" id="rollno" name="rollno" required>
    <option selected>Select RollNo</option>
    {% for r in query %}
    <option value="{{r.rollno}}">{{r.rollno}}</option>
    {% endfor %}
</select>
</div>
<br>
<div class="form-group">
<label for="attendance">Attendance Percentage</label>
<input type="number" class="form-control" name="attend" id="attend">
</div>
<br>
<button type="submit" class="btn btn-danger btn-sm btn-block">Add
Attendance</button>

```

```

</form>

<br>

<br>

</div>

<div class="col-md-4"></div>

</div></div>

{% endblock body %}

```

TRIGGERS

```

{% extends 'base.html' %}

{% block title %}

Triggers

{% endblock title %}

{% block body %}

<h3 class="text-center"><span>Student Triggers Records</span> </h3>

{% with messages=get_flashed_messages(with_categories=true) %}

{% if messages %}

{% for category, message in messages %}

<div class="alert alert-{{category}} alert-dismissible fade show" role="alert">

    {{message}}

</div>

{% endfor %}

{% endif %}

{% endwith %}

<br>

<div class="container mt-4">

<table class="table">

    <thead class="thead-light">

        <tr>

            <th scope="col">TID</th>

            <th scope="col">ROLL NUMBER</th>

            <th scope="col">ACTION</th>

            <th scope="col">TIMESTAMP</th>

        </tr>

```

```

</thead>
<tbody>
{% for post in query %}
<tr>
<th scope="row">{{post.tid}}</th>
<td>{{post.rollno}}</td>
<td>{{post.action}}</td>
<td>{{post.timestamp}}</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% endblock body %}

```

Css code

1. PHONE REGISTER

```

.form-group input {
width: 100%;
border: none;
border-bottom: 1px solid #6617cb;
outline: none;
font-size: 20px;
margin-bottom: 3px;
}
.drop {
width: 100%;
border: none;
border-bottom: 1px solid orangered;
outline: none;
font-size: 16px;
margin-bottom: 5px;
background: white;
}

```

```

textarea {
  margin-top: 0px;
  margin-bottom: 0px;
  height: 120px;
  width: 100%;
}
.form-group input: hover {
  background-color: gainsboro
}

```

2. Virtual register

```

.section-title {
  text-align: center;
  /* padding-bottom: 30px; */
}
.pricing .row {
  padding-top: 0px;
}
h6 {
  font-family: 'Cormorant Garamond', serif;
  color: black;
  text-align: center;
  margin-top: 10px;
  border-radius: 15px;
  font-size: 22px;
  position: relative;
  animation-name: sign;
  animation-duration: 3s;
  animation-iteration-count: infinite;
}
@keyframes sign {
  0% {
    top: 0px;
    background-color: #ff4e00;

```



```

    background-image: linear-gradient(315deg, #ff4e00 0%, #ec9f05 74%);
}
50% {
    top: 10px;
    background-color: #ff4e00;
    background-image: linear-gradient(315deg, #ff4e00 0%, #ec9f05 74%);
}
100% {
    top: 10px;
}
}
.card {
    margin-top: 57px;
    border-radius: 8px 40px 40px 8px;
    transition: all 0.5s ease-in-out;
    box-shadow: 1px 0px 8px 2px orangered;
}
label {
    color: black;
    margin-top: 15px;
    font-family: 'Itim', cursive;
}
.form-group input {
    width: 400px;
    border: none;
    /* border-bottom: 1px solid purple; */
    border-bottom: 2px solid red;
    outline: none;
    font-size: 22px;
    margin-bottom: 5px;
    font-family: 'Itim', cursive;
}
textarea {
    resize: vertical;

```

```

    width: 400px;
    border-bottom: 2px solid red;
}
.drop {
    width: 400px;
    border: none;
    border-bottom: 2px solid red;
    outline: none;
    font-size: 22px;
    margin-bottom: 5px;
}
.form-group input: hover {
    background-color: ghostwhite;
}
#btn {
    background-color: #7f5a83;
    background-image: linear-gradient(315deg, #7f5a83 0%, #0d324d 74%);
    border-radius: 15px;
    border: none;
    position: relative;
}
#btn: hover {
    background-color: #0cbaba;
    background-image: linear-gradient(315deg, #0cbaba 0%, #380036 74%);
}

```

CHAPTER-5

DEPLOYMENT

The deployment of the Institute Management System was designed to ensure portability, scalability, and reproducibility using modern DevOps practices. Instead of running the application directly on a host system, the entire application stack was containerized using Docker and orchestrated using Docker Compose.

5.1 Deployment Architecture

The system follows a **multi-tier architecture** consisting of three main services:

- **Frontend Access Layer:** Nginx (Reverse Proxy)
- **Application Layer:** Flask (via Gunicorn)
- **Database Layer:** MySQL (MariaDB)

Architecture Flow Diagram



Fig. 5.1 Deployment architecture flow diagram

Nginx handles incoming HTTP requests and forwards them to the Flask application running through Gunicorn.

5.2 Containerization Using Docker

Docker was used to package the application and its dependencies into containers. Each service runs in an isolated container:

Flask Application Container

- Built using a custom **Dockerfile**
- Includes Python runtime and required dependencies
- Runs using **Gunicorn** for production-level performance

MySQL Database Container

- Uses official **MariaDB image**
- Automatically initializes the database using an SQL dump file
- Stores persistent data using Docker volumes

Nginx Container

- Acts as a **reverse proxy**
- Routes client requests to the Flask backend
- Improves performance and security

Orchestration Using Docker Compose

Docker Compose was used to manage and run multiple containers simultaneously.

Key Features:

- Defines all services in a single `docker-compose.yml` file
- Automatically creates a shared network
- Enables communication between services using service names
- Allows one-command deployment:

Reverse Proxy Configuration (Nginx)

Nginx acts as an intermediary between the client and the backend application.

Functions:

- Handles incoming HTTP requests
- Routes traffic to the Flask application
- Improves performance through efficient request handling

- Adds an extra layer of security

Reproducibility and Portability

One of the key goals of deployment was to make the project easy to run on any system.

Steps to Run:

1. Install Docker and Docker Compose
2. Clone the repository
3. Run: `docker compose up -d`

The application will start automatically without any manual configuration.

Deployment on AWS EC2

To make the application accessible over the internet, it was deployed on an **AWS EC2 instance**, which provides a virtual server in the cloud.

Steps Involved:

➤ Launch EC2 Instance

- Selected Ubuntu-based instance
- Configured key pair for secure login

➤ Configure Security Group

- Allowed inbound traffic on:
 - Port 22 (SSH)
 - Port 80 (HTTP)

➤ Connect to Instance

`ssh ubuntu@<public-ip>`

Install Docker & Docker Compose

- Installed required packages on EC2

➤ Transfer Project Files

- Cloned project from GitHub repository

➤ Run Application

`docker compose up -d`

➤ Access Application

Opened in browser using EC2 public IP: `http://<public-ip>`

5.3 AI Involvement in Maintenance & Upgradation

During the maintenance and upgradation phase of the application, modern AI-powered development tools such as GitHub Copilot and OpenAI Codex were extensively utilized. These tools played a significant role in improving both the functionality and the user experience of the system.

Initially, the application had a very basic and minimal user interface, which lacked interactivity and did not align with current industry standards. By leveraging AI assistance, we were able to redesign and enhance the UI to make it more intuitive, responsive, and visually appealing. The AI tools helped generate modern UI components, suggest better layouts, and improve styling practices, resulting in a more user-friendly and professional-looking application. In addition to UI improvements, AI was also instrumental in resolving a critical data integrity issue. The system previously allowed duplicate entries of students within the same department and semester using identical roll numbers. This flaw could lead to inconsistencies and errors in data management. With the help of AI-generated suggestions and code validation techniques, we implemented appropriate constraints and logic checks in the backend. As a result, the system now enforces uniqueness, ensuring that no two students can be registered with the same roll number in the same department and semester. This significantly improved the reliability and accuracy of the application.

Chapter 6

Conclusion

The Institute Management System (IMS) has been successfully designed and developed to computerize and automate the major activities of an educational institute. The main objective of this project was to replace the traditional manual system with a modern, efficient, secure, and user-friendly web-based solution. The developed system provides a centralized platform where administrators can manage student records, departments, attendance, and user authentication efficiently. The project has demonstrated how modern web technologies such as Python Flask, MySQL, HTML, CSS, JavaScript, and Bootstrap can be used to build a reliable and scalable management system. The system overcomes the limitations of the existing manual system by reducing paperwork, minimizing data redundancy, improving data accuracy, and enabling quick retrieval of information. The use of authentication and password security ensures that the data remains protected and accessible only to authorized users. The implementation of triggers and stored procedures also improves database integrity and tracking of operations. This project has also provided practical exposure to real-world software development practices such as requirement analysis, database design, implementation, testing, and deployment. It demonstrates the importance of automation in educational institutions and how digital systems can significantly improve administrative efficiency and decision-making. Although the current system fulfills the basic requirements of student and institute management, there is significant scope for further enhancement by integrating advanced features such as online fee payment, result management, mobile application support, biometric attendance, cloud deployment, and AI-based analytics. In addition, the system can be further optimized for performance and scalability to handle larger datasets and multiple users simultaneously. Future improvements may also focus on enhancing user experience through better UI/UX design and accessibility features. The adoption of such systems can contribute to building a fully digital and paperless environment within institutions. Moreover, continuous updates and maintenance will ensure that the system remains relevant to evolving technological trends. In conclusion, the Institute Management System is a reliable, efficient, and scalable solution that simplifies institute operations and lays a strong foundation for the development of a complete Smart Campus ERP system in the future.

6.1 Future Enhancements

The Institute Management System can be further improved by incorporating several advanced features to enhance its functionality and usability. In the future, the system can be upgraded with a more interactive and responsive user interface to provide a better user experience across different devices, including mobile phones and tablets. Integration of role-based authentication with enhanced security features, such as OTP verification and data encryption, can be implemented to ensure data privacy and protection. The system can also be extended by adding modules like fee management, examination management, result processing, and timetable scheduling to make it more comprehensive. Additionally, implementing real-time notifications through email or SMS can help keep students and administrators updated about important activities such as attendance alerts and announcements. The use of cloud technology can improve scalability and allow remote access to data, while incorporating data analytics and reporting features can help institutions make better decisions. Furthermore, AI-based features such as automated attendance using facial recognition and predictive analysis for student performance can make the system more intelligent and efficient.

In addition to these, the system can be integrated with mobile applications to provide anytime, anywhere access for users. A dashboard with graphical representations of data can be introduced for better visualization and monitoring of student performance and institutional activities. Integration with third-party services such as payment gateways can simplify fee transactions. The system can also include backup and recovery mechanisms to prevent data loss. Moreover, multi-language support can be added to make the system accessible to a wider range of users. Overall, these enhancements will transform the system into a more robust, secure, and future-ready solution.

6.2 Screenshots of the application

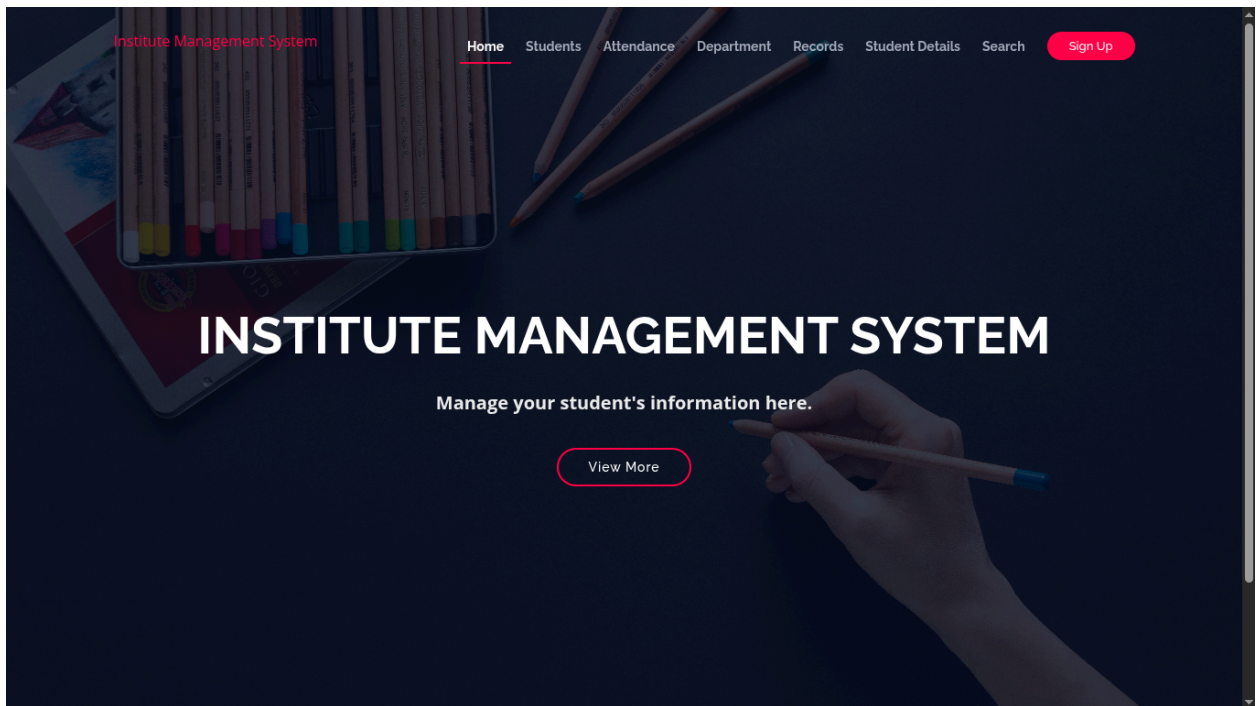


Fig 6.1 Home page

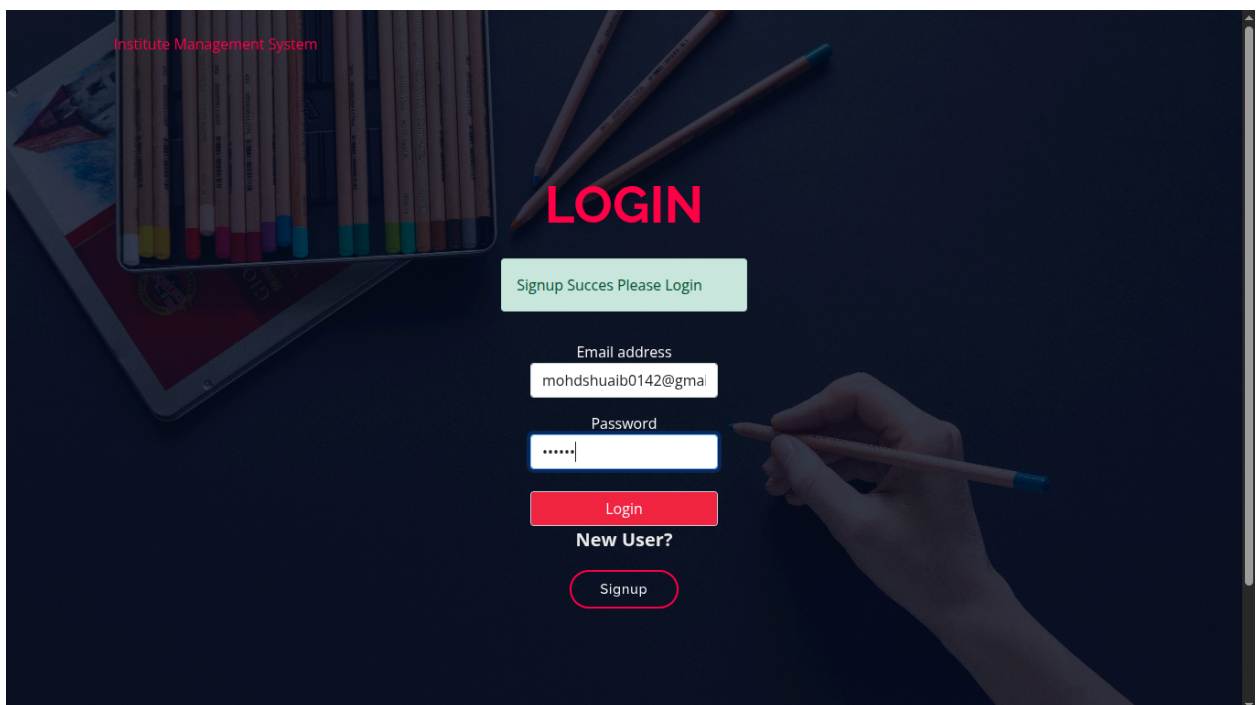


Fig 6.2 Login page

Institute Management System

HomeStudentsAttendanceDepartmentRecordsStudent DetailsSearchWelcome Mohd ShuaibLogout

Add Student Details

Roll Number

2416080003

Student Name

Vaibhav Arya

Sem

4

Male

computer science

Email

aryavaibhav245@gmail

Phone Number

7668773747

Address

Saharanpur Uttar Pradesh India

Add Record

Fig 6.3 Adding Student

Institute Management System

HomeStudentsAttendanceDepartmentRecordsStudent DetailsSearchWelcome Mohd ShuaibLogout

INSTITUTE MANAGEMENT SYSTEM

Manage your student's information here.

View More

Add Department

Enter Department

Information Technology

Add Department

Fig 6.4 Adding department

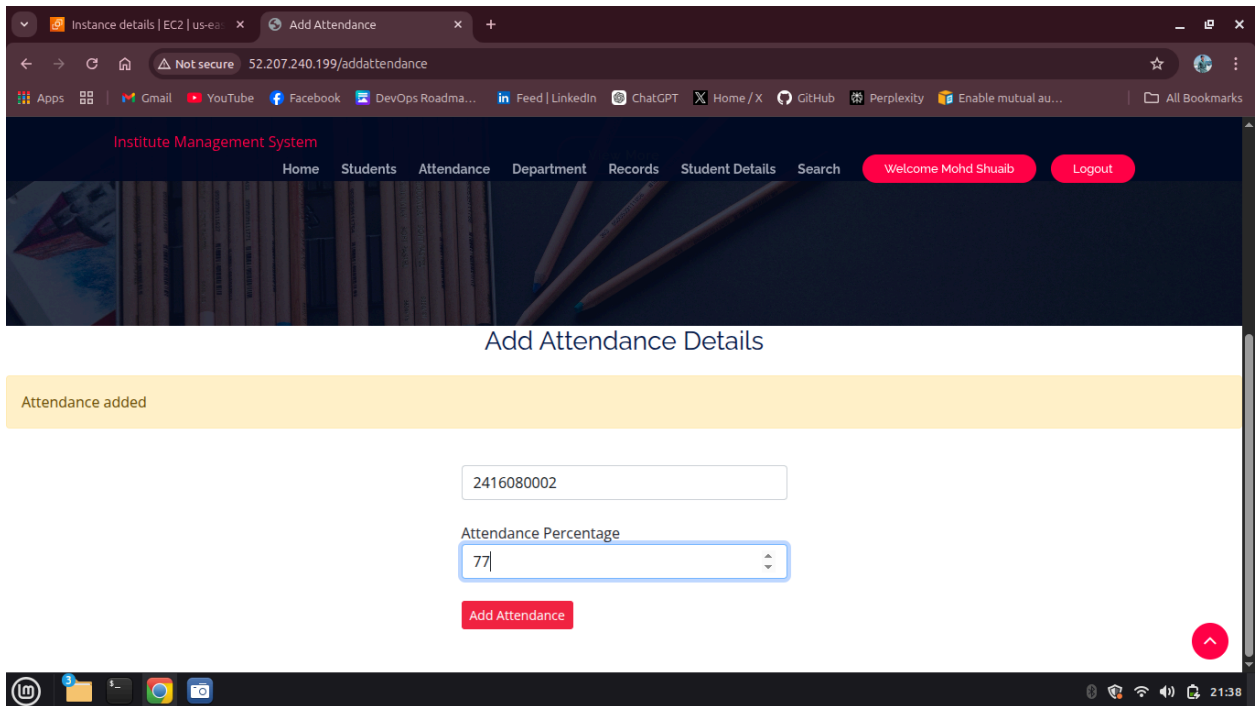


Fig 6.5 Adding Attendance

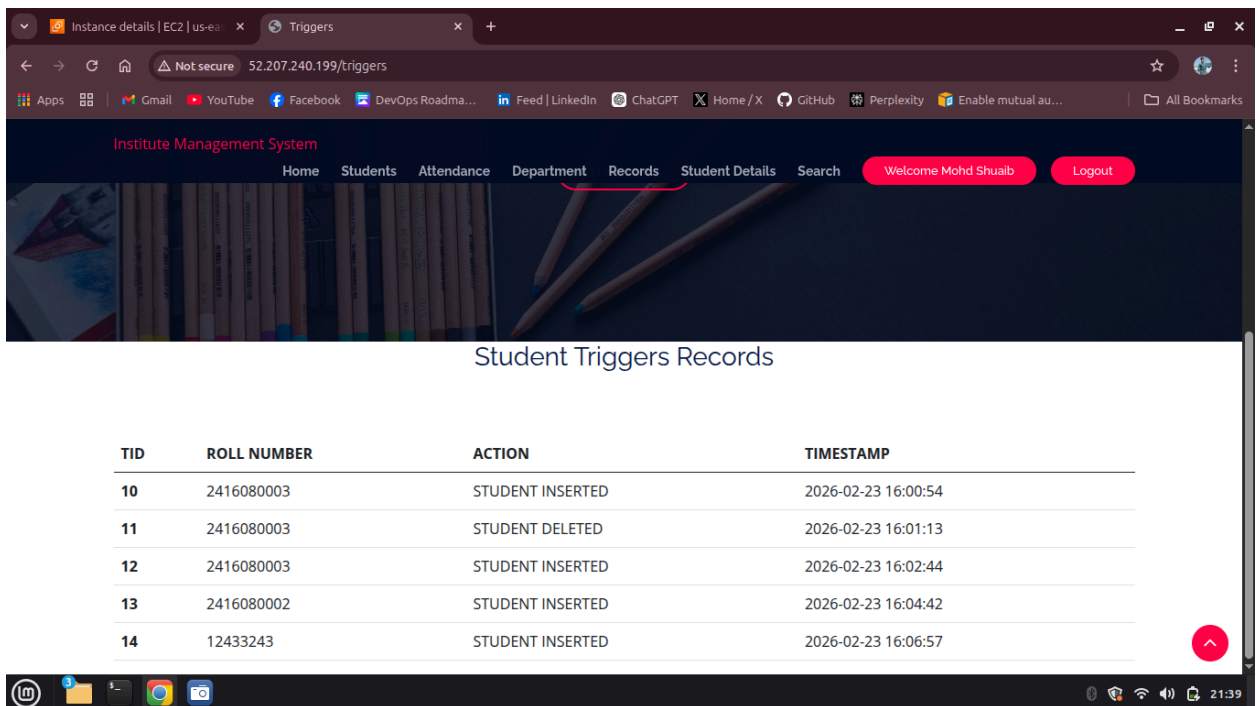


Fig 6.6 Trigger

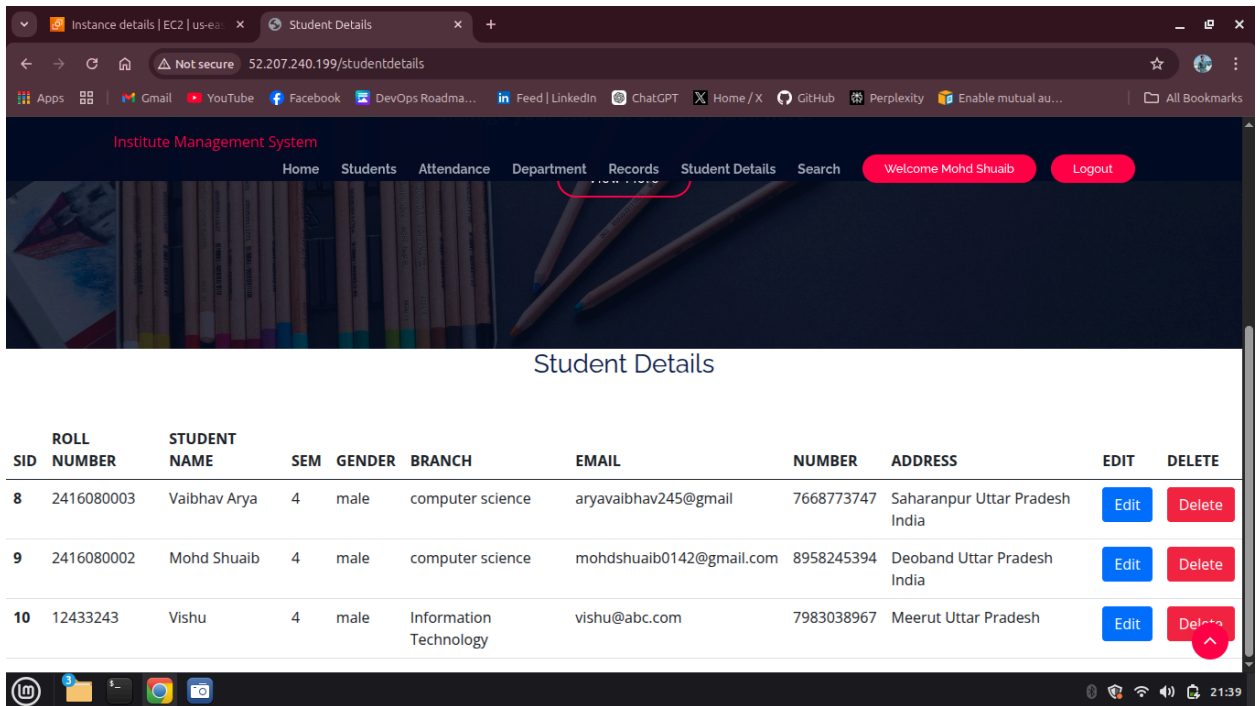


Fig 6.7 Student details

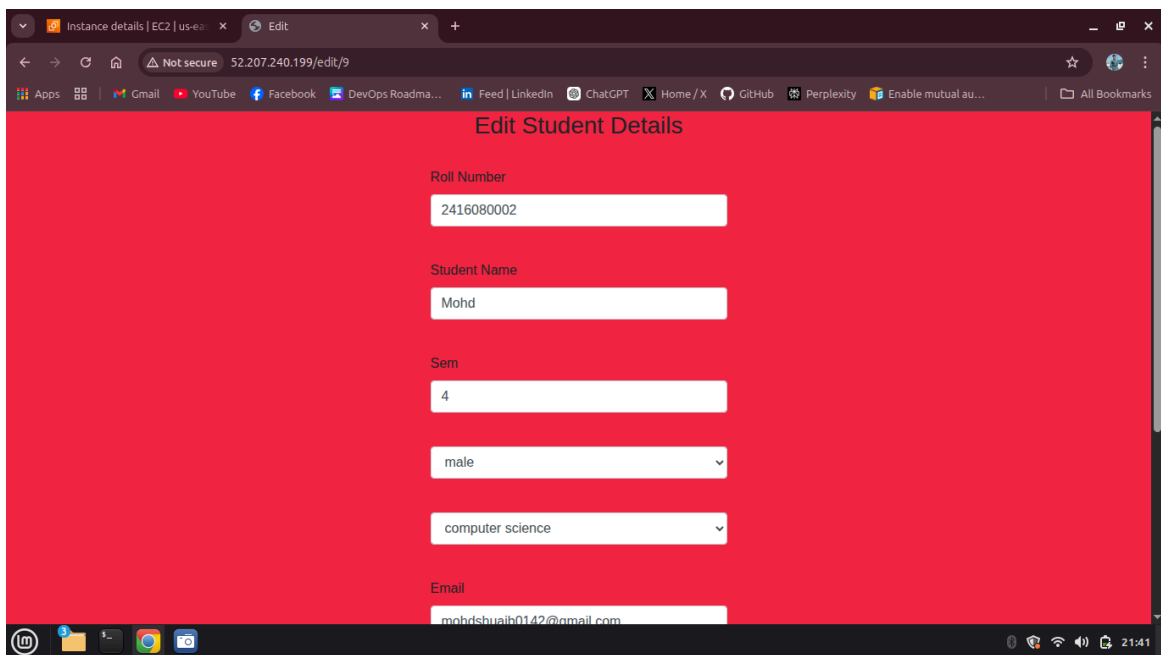


Fig 6.8 Edit Student detail

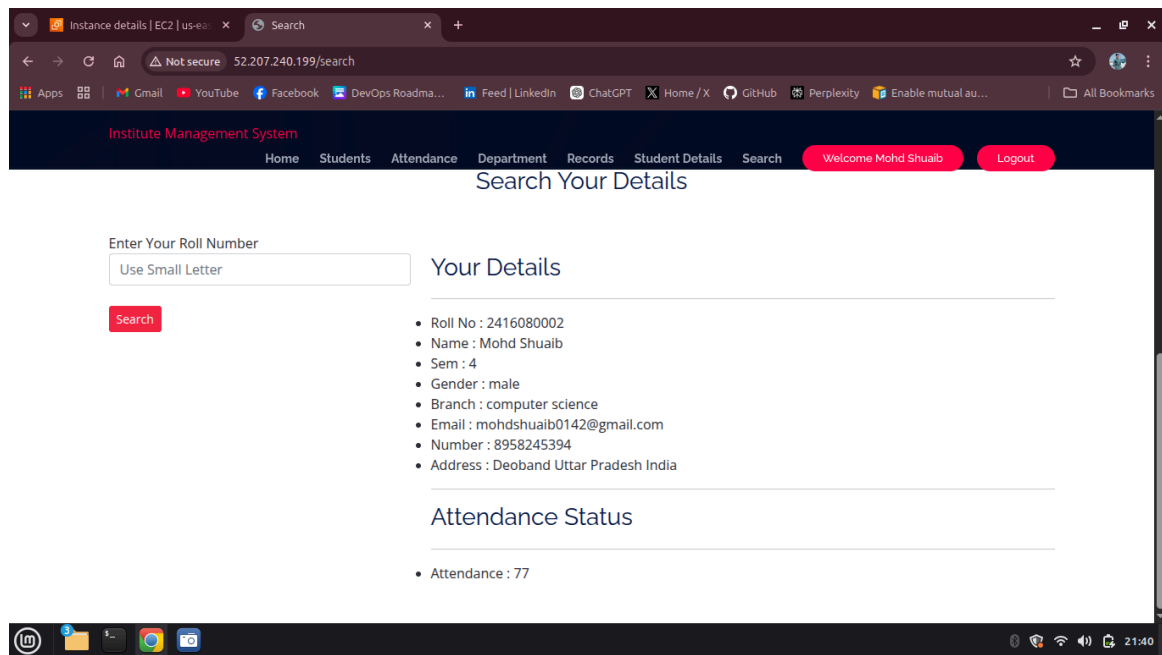


Fig 6.9 Search Student detail

References

- Fundamentals of Database Systems – Ramez Elmasri and Shamkant Navathe
- Software Engineering – K. K. Aggarwal and Yogesh Singh
- Google – <https://www.google.com>
- GeeksforGeeks – <https://www.geeksforgeeks.org>
- GitHub – <https://github.com>
- ChatGPT – <https://chatgpt.com>
- Medium – <https://medium.com>
- W3Schools – <https://www.w3schools.com>
- Stack Overflow – <https://stackoverflow.com>
- TutorialsPoint – <https://www.tutorialspoint.com>
- freeCodeCamp – <https://www.freecodecamp.org>